

Ball Machine

Madina heeft een ballenmachine uitgevonden om de deelnemers van de IOI te vermaken. De machine werkt als volgt:

- De machine bestaat uit N **knopen**, geïndexeerd van 0 tot en met $N - 1$. Knoop $N - 1$ heet de **wortel**.
- Voor elke knoop u met $0 \leq u < N - 1$, is de knoop $P[u]$ de **ouder** van knoop u , met $P[u] > u$, en knoop u is een **kind** van $P[u]$. De wortel heeft geen ouder. Een knoop zonder kinderen heet een **blaadje**.

De waarde van N is **niet** bekend, net zoals de ouder $P[u]$ van alle knopen u . In plaats daarvan vertelt Madina je M , het aantal blaadjes, en dat de indices van de blaadjes $0, 1, \dots, M - 1$ zijn. Jouw taak is om de interne structuur van de machine te bepalen door hem te gebruiken.

Elke knoop van de machine kan maximaal 1 **bal** bevatten, en elke bal heeft een **waarde** die een niet-negatief getal is. We zeggen dat een knoop waarde x heeft als hij een bal bevat met waarde x . Een knoop is **bezet** als hij een bal bevat; anders is hij **vrij**. Elke knoop begint vrij.

Je kan twee soorten operaties doen:

- **insert**(U, X): probeer een nieuwe bal met waarde X in de machine te stoppen bij blaadje U .
 - Als blaadje U bezet is geeft de operatie `false` terug zonder de bal erin te stoppen.
 - Als blaadje U vrij is, wordt de bal bij blaadje U geplaatst. Dan beweegt de bal steeds naar de ouder van zijn huidige knoop, zolang die ouder vrij is. De beweging stopt wanneer de bal de wortel bereikt of als de ouder bezet is. De operatie geeft `true` terug.
- **collect**(**)**: als de wortel vrij is (wat betekent dat machine leeg is), geeft `collect` een lege lijst terug. Anders gaat de procedure `traverse`, die hieronder beschreven wordt, de waardes van alle ballen in de machine verzamelen in een lijst S . De lijst S begint leeg, en `traverse(N - 1)` wordt aangeroepen.

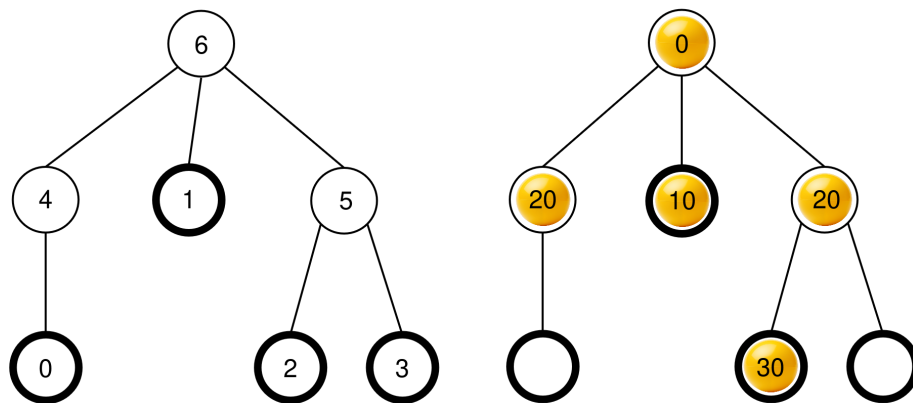
```

traverse(u) :
  voeg de waarde van de bal bij knoop u toe aan het einde van  $S$ 
  laat  $c[u]$  de lijst zijn met bezette kinderen van u
  sorteer  $c[u]$  in niet-dalende volgorde van de waardes
    van de ballen bij die knopen (als meerdere knopen
    dezelfde waardes hebben, kunnen ze in elke
    volgorde voorkomen)
  voor elke  $v$  in  $c[u]$ :
    traverse(v)

```

Zodra deze procedure klaar is worden **alle ballen verwijderd** uit de machine, en geeft `collect` S terug.

Bekijk bijvoorbeeld een machine met $N = 7$ knopen en $P = [4, 6, 5, 5, 6, 6]$ de lijst van ouders. De afbeelding links geeft de lege machine weer met de indices van de knopen, en de afbeelding rechts laat een mogelijke configuratie van de ballen zien na een serie van `insert` operaties (deze serie van operaties wordt gegeven in de Voorbeeld sectie).



Wanneer `collect()` aangeroepen wordt is de wortel (knoop 6) bezet, dus S wordt geïnitieerd als $S = []$ en `traverse(6)` wordt aangeroepen.

traverse(6): knoop 6 heeft waarde 0; als die toegevoegd wordt aan S krijgen we $S = [0]$. De kinderen van knoop 6 zijn 4, 1, en 5, die allemaal bezet zijn. De waardes zijn respectievelijk 20, 10, en 20. Als we dat sorteren in niet-dalende volgorde krijgen we $c[6] = [1, 5, 4]$ (merk op dat $c[6] = [1, 4, 5]$ ook mogelijk is, aangezien 4 and 5 dezelfde waarde hebben). Dan roept de procedure `traverse` aan op elke knoop in $c[6]$ in die volgorde.

- **traverse(1):** knoop 1 heeft waarde 10; toevoegen aan S geeft $S = [0, 10]$. Knoop 1 heeft geen bezette kinderen, dus deze aanroep eindigt.
- **traverse(5):** knoop 5 heeft waarde 20; toevoegen aan S geeft $S = [0, 10, 20]$. Knoop 5 heeft één bezet kind, knoop 2, dus $c[5] = [2]$ en deze procedure roept `traverse(2)` aan.

- **traverse(2)**: knoop 2 heeft waarde 30; toevoegen aan S geeft $S = [0, 10, 20, 30]$. Knoop 2 heeft geen bezette kinderen, dus deze aanroep eindigt.
- De uitvoering hervat in **traverse(5)**, waar alle kinderen van $c[5]$ afgehandeld zijn, dus de aanroep eindigt.
- **traverse(4)**: knoop 4 heeft waarde 20; toevoegen aan S geeft $S = [0, 10, 20, 30, 20]$. Knoop 4 heeft geen bezette kinderen, dus deze aanroep eindigt.

Alle aanroepen zijn klaar en er zijn geen knopen meer om te verwerken. Uiteindelijk wordt elke bal uit de machine verwijderd en geeft **collect** de lijst $S = [0, 10, 20, 30, 20]$ terug.

Jouw taak is om het aantal knopen N te bepalen, en een lijst van ouders $R = [R[0], R[1], \dots, R[N-2]]$ terug te geven die de structuur van de machine beschrijft, mogelijk met een andere labeling (van de knopen die niet de blaadjes zijn en niet de wortel). Formeel gezien is een lijst R correct als het mogelijk is om een uniek label $L[u]$ met $0 \leq L[u] < N$ aan elke knoop u van de machine te geven zodat:

- $L[u] = u$ voor alle $0 \leq u < M$ en $u = N - 1$, en
- $L[P[u]] = R[L[u]]$ voor alle $0 \leq u < N - 1$.

Het is **niet** nodig dat $R[u] > u$. De machine **moet leeg zijn** wanneer je jouw oplossing teruggeeft.

Laat K het aantal **collect** operaties zijn dat gedaan is, en B de maximale waarde van alle ballen over alle aanroepen van **insert**. Neem $C = K + B$. Dan **moet C niet groter zijn dan 1000**. Je score in sommige subtaken hangt af van de waarde van C .

Implementatiedetails

Je moet de volgende procedure implementeren.

```
std::vector<int> find_structure(int M)
```

- M : het aantal blaadjes in de machine.
- De procedure wordt precies één keer aangeroepen voor elk testgeval.
- De procedure moet een lijst $R = [R[0], R[1], \dots, R[N-2]]$ van lengte $N - 1$ teruggeven, een correcte lijst van ouders.

Om de machine te gebruiken kan jouw procedure de volgende twee procedures gebruiken.

De eerste procedure is:

```
bool insert(int U, int X)
```

- U : de index van het blaadje waar de bal ingevoegd zou moeten worden. Die moet voldoen aan $0 \leq U < M$.
- X : de waarde van de bal. Die moet voldoen aan $0 \leq X \leq 1000$.
- De procedure geeft `true` terug als de bal succesvol toegevoegd is, of `false` als blaadje U al bezet was.
- Deze procedure mag maximaal 500 000 keer aangeroepen worden.

De tweede procedure is:

```
std::vector<int> collect()
```

- Verzamelt de waardes van alle toegevoegde ballen, leegt de machine, en geeft de lijst S terug.

Als de waarde van C op enig moment groter wordt dan 1000, krijgt de oplossing het resultaat `Output isn't correct: Too many resources used.`

De structuur van de machine staat vast voordat `find_structure` aangeroepen wordt. De grader is **deterministisch** in de zin dat als je het twee keer aanroept en and beide aanroepen dezelfde serie van operaties doet, dan geven de aanroepen naar `collect` dezelfde lijsten terug.

Beperkingen

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtaken

Subtaak	Score	Extra beperkingen
1	5	De wortel heeft precies M kinderen.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Geen extra beperkingen.

In subtaak 3 en 4 hangt je score af van de waarde van C als volgt.

Subtaak 3 (25 points)

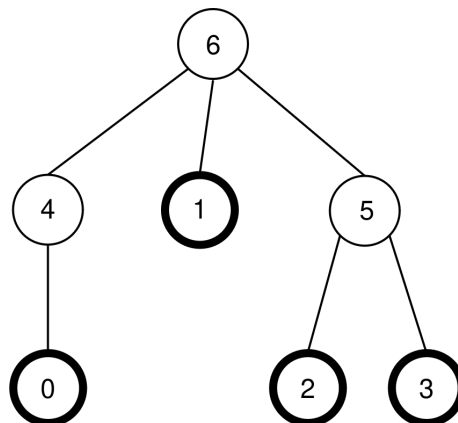
Limieten	Score
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtaak 4 (60 points)

Limieten	Score
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Voorbeeld

Bekijk dezelfde machine als in de taakbeschrijving, dus $N = 7$, $M = 4$, en $P = [4, 6, 5, 5, 6, 6]$. De machine wordt opnieuw weergegeven in de volgende afbeelding:



De grader doet de volgende aanroep:

```
find_structure(4)
```

De procedure kan de volgende serie van aanroepen doen:

1. **insert(0, 0)**: blaadje 0 is vrij, dus een bal met waarde 0 wordt bij blaadje 0 geplaatst. Knoop $P[0] = 4$ is vrij, dus de bal beweegt naar knoop 4. Knoop $P[4] = 6$ is vrij, dus de bal

beweegt naar knoop 6. Omdat knoop 6 de wortel is, stopt de beweging. De aanroep geeft `true` terug.

2. `insert(3, 20)`: blaadje 3 is vrij, dus een bal met waarde 20 wordt bij blaadje 3 geplaatst. Knoop $P[3] = 5$ is vrij, dus de bal beweegt naar knoop 5. Omdat $P[5] = 6$ vrij is, stopt de beweging. De aanroep geeft `true` terug.

3. `insert(1, 10)`: blaadje 1 is vrij, dus een ball met waarde 10 wordt geplaatst bij blaadje 1. Omdat $P[1] = 6$ bezet is, blijft de bal bij knoop 1. De aanroep geeft `true` terug.

4. `insert(2, 30)`: blaadje 2 is vrij, dus een bal met waarde 30 wordt geplaatst bij blaadje 2. Omdat $P[2] = 5$ bezet is, blijft de bal bij knoop 2. De aanroep geeft `true` terug.

5. `insert(1, 25)`: blaadje 1 is bezet, dus de bal wordt niet bij blaadje 1 geplaatst. De aanroep geeft `false` terug.

6. `insert(0, 20)`: blaadje 0 is vrij, dus de bal met waarde 20 wordt geplaatst bij waarde 0. Blaadje $P[0] = 4$ is vrij, dus de bal verplaatst naar knoop 4. Omdat $P[4] = 6$ bezet is, stopt de beweging. De aanroep geeft `true` terug.

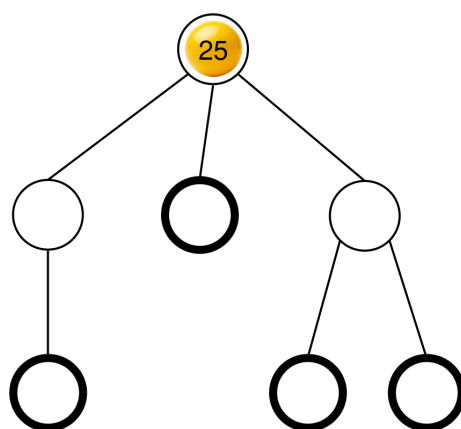
De resulterende configuratie van ballen is al in de taakbeschrijving te zien.

Daarna kan de procedure aanroepen:

```
collect()
```

De uitvoer van deze aanroep is al uitgelegd in de taakbeschrijving, dus deze aanroep geeft $S = [0, 10, 20, 30, 20]$ terug. Daarna is de machine weer leeg.

Dan kan de procedure `insert(2, 25)`. Blaadje 2 is vrij, dus een bal met waarde 25 wordt geplaatst bij blaadje 0. Deze bal beweegt naar knoop 5, en vervolgens naar knoop 6, waar hij stopt. De aanroep geeft `true` terug. The resulterende configuratie van ballen is weergegeven in de volgende afbeelding.



Uiteindelijk kan de procedure `collect()` aanroepen, wat $S = [25]$ teruggeeft en de machine leegt

Er zijn twee mogelijke lijsten van ouders die `find_structure` kan teruggeven: $R = P = [4, 6, 5, 5, 6, 6]$ (wat correspondeert met de labeling $L = [0, 1, 2, 3, 4, 5, 6]$), en $R = [5, 6, 4, 4, 6, 6]$ (wat corresponds met de labeling $L = [0, 1, 2, 3, 5, 4, 6]$). In dit voorbeeld hebben we $K = 2$ en $B = 30$, dus $C = 32$.

Voorbeeldgrader

Invoerformaat:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Uitvoerformaat:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Hier is Q de lengte van de lijst R teruggegeven door `find_structure`.